

The Project, Explained Simply

Everything done so far, every decision and **why** it was made, and the questions you may be asked — written in plain words so you can explain it to anyone.

PROJECT

Predictive Cloud Resource Optimizer

DATA

Bitbrains fastStorage — 1,250 real servers

COVERS

Steps 1.1, 1.2 and 1.3

CHOSEN SERVER

VM 270

How to use this guide. Read Part 1 and Part 2 first — they give you the big idea and the words you need. After that, each part covers one piece of work. Every decision is written as **What we chose, Why**, and **What goes wrong if we don't**. Part 10 has likely teacher questions with short answers.

1 The big idea

In one line: We predict how busy a server will be 5 minutes from now, so we can add more computers *before* it gets overloaded, not after.

The problem

Websites and apps run on servers in the cloud. The number of users keeps changing. When many users come at once (a "spike"), the server gets overloaded and the app becomes slow.

The usual fix is **auto-scaling**: when the server gets too busy, the system starts another server to share the work. Most systems do this **reactively** — they wait until CPU crosses a limit (for example 75%) and then react.

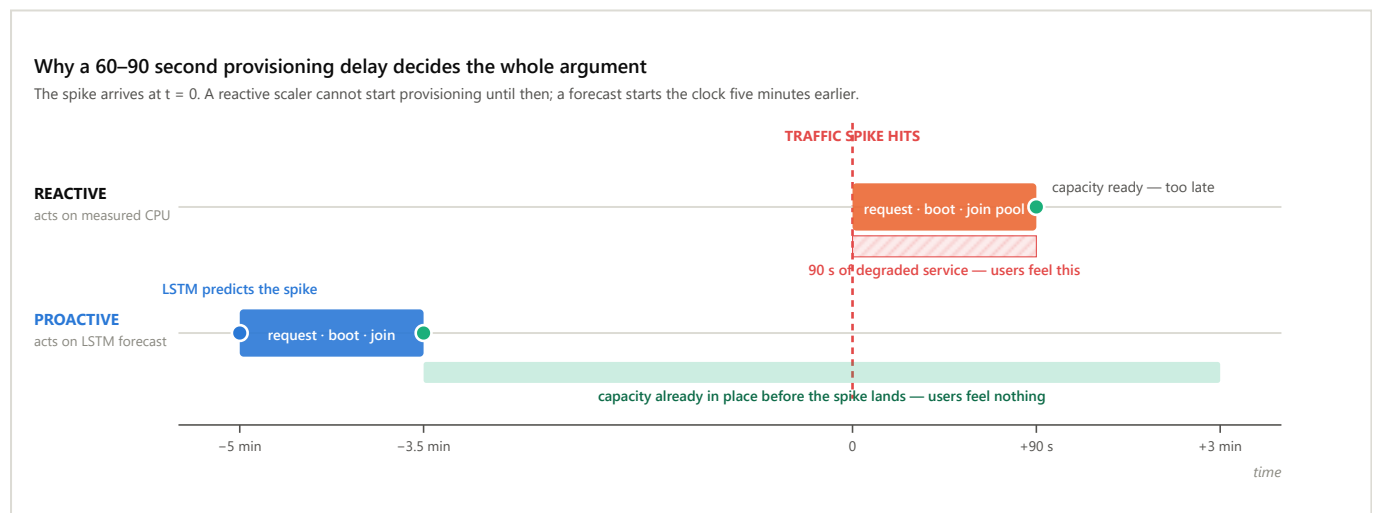
The trouble: a new server takes about **60 to 90 seconds** to start and become ready. So by the time help arrives, users have already suffered the slowdown.

A SIMPLE EXAMPLE

Think of a shop. A **reactive** manager calls extra staff only when the queue is already long — the staff take 15 minutes to arrive, so customers wait. A **proactive** manager knows the lunch rush starts at 1 pm and calls staff at 12:45 — they are ready when the rush begins. Our project is the proactive manager.

Our solution

We use an **LSTM** — a type of neural network that is good at learning from data that changes over time. It looks at the last 5 hours of server activity and predicts the next 5 minutes. If it predicts a spike, we add servers early.



What we must prove at the end

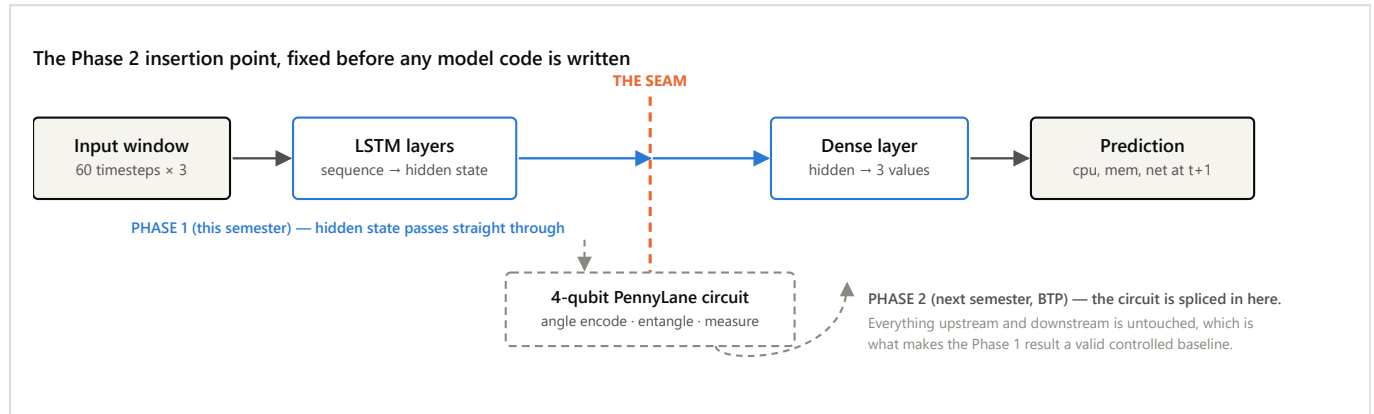
The whole project exists to produce one comparison: **proactive scaling** (using our prediction) versus **reactive scaling** (using the current reading). We will measure:

- **Spike response time** — how quickly extra capacity is ready when a spike comes.
- **Over-provisioning** — how much extra capacity we paid for but did not need.
- **False positives** — how often we added servers for a spike that never came.

Two phases

- **Phase 1 (this semester):** a complete normal (classical) system: data → LSTM → scaling → dashboard.
- **Phase 2 (next semester):** add a small **quantum circuit** (4 qubits, using PennyLane) inside the model, and compare it with Phase 1.

Because Phase 2 will be compared with Phase 1, Phase 1 must be clean, correct and never thrown away. The quantum part will be inserted at one fixed point in the model:



2 Words you need to know

Read these once. Every later part uses them.

VM (virtual machine)	A "computer inside a computer". One big physical server is split into many VMs. In our dataset, each file is one VM.
CPU usage %	How hard the processor is working. 0% = doing nothing, 100% = fully busy.
Memory usage %	How much of the RAM given to the VM is in use.
Network (KB/s)	How much data goes in and out of the VM each second.
Time series	Readings taken one after another in time. Ours has one reading every 5 minutes.
Mean	The average value. Add all readings and divide by how many there are.
Standard deviation (std)	How much the values spread out from the mean. Small std = values stay close to the average (flat). Big std = values swing up and down a lot.
Gaussian / bell curve	A bell-shaped picture of spread, where most values sit near the middle. Many datasets look like this; ours does not , so our graphs show the real readings instead (see Part 4).
LSTM	"Long Short-Term Memory". A neural network that reads a sequence step by step and remembers what matters, so it can predict what comes next.
Training / validation / test data	Training: data the model learns from. Validation: data used while training to check it is not just memorising. Test: data kept hidden until the very end, for the final honest score.

Scaling (normalisation)	Converting all numbers to the same range, here 0 to 1, so no column looks "more important" just because its numbers are bigger.
Scaler	The object that remembers the minimum and maximum used for scaling, so we can convert predictions back to real units later.
Window	A slice of consecutive readings given to the model as one example. Ours is 60 readings = 5 hours.
Batch	A group of windows given to the model together. Ours is 64 windows per batch.
Data leakage	When information from the test data sneaks into training. The model then looks better than it really is — like seeing exam answers before the exam.
Tensor / DataLoader	A tensor is PyTorch's version of an array. A DataLoader hands tensors to the model one batch at a time.
Resampling	Putting readings onto an exact, regular time grid (here every 5 minutes).

THE MOST IMPORTANT WORD: STANDARD DEVIATION

Two VMs can both have a mean CPU of 50%. One stays at 50% all day (std close to 0). The other jumps between 0% and 100% (std close to 50). Only the second one has **changes** for a model to learn and predict. That is why std is the main number we used to pick a VM.

3 The data

In one line: Real monitoring data from 1,250 business servers, one reading every 5 minutes.

Where it comes from

Bitbrains was a Dutch hosting company. Its customers were banks and insurance companies running heavy jobs like financial reports and risk calculations. Researchers at **Delft University (TU Delft)** collected real readings from these servers, removed private details, and published them as the **GWA-T-12 Bitbrains** dataset. We use the **fastStorage** part: 1,250 VMs.

CREDIT WE MUST GIVE IN THE REPORT

Shen, van Beek & Iosup, "Statistical Characterization of Business-Critical Workloads Hosted in Cloud Datacenters", CCGrid 2015. Data courtesy of Bitbrains IT Services.

What is inside one file

Each VM is one CSV file with 11 columns, separated by **semicolons (;)**, not commas. We keep only what we need and turn it into 4 simple columns:

From eleven raw columns to the four-column canonical format

RAW BITBRAINS CSV (semicolon-delimited)

Timestamp [ms]	actually Unix seconds
CPU-cores	
CPU-capacity-provisioned [MHz]	
CPU-usage [MHz]	
CPU usage [%]	
Memory capacity provisioned [KB]	denominator
Memory usage [KB]	numerator
Disk read throughput [KB/s]	
Disk write throughput [KB/s]	
Network received throughput [KB/s]	
Network transmitted throughput [KB/s]	

CANONICAL OUTPUT

datetime

index — strict 5-minute steps

cpu

CPU usage [%], clipped to 0–100

mem

used / capacity × 100, clipped

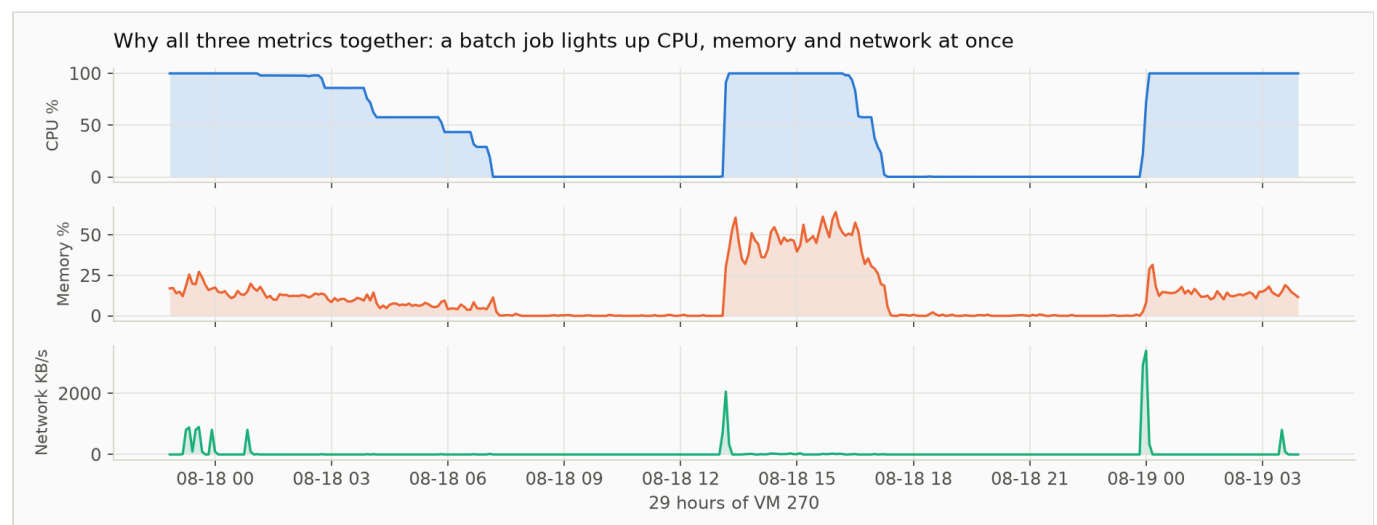
net

received + transmitted

Six columns discarded: CPU core count and MHz capacity are static allocation figures, not load; the disk columns are left out because the project forecasts and scales only CPU, memory and network.

Why these three measurements

When a big job starts on a server, three things usually rise together: the **CPU** (the calculation runs), the **memory** (data is loaded), and the **network** (data is fetched or sent). They do not rise at exactly the same moment, but they are connected. A model that sees all three together can learn these connections, so it has more information than a model that sees CPU alone.



Picture. About 29 hours of VM 270. Look at just after midnight on the right: CPU, memory and network all jump at the same time when a job starts.

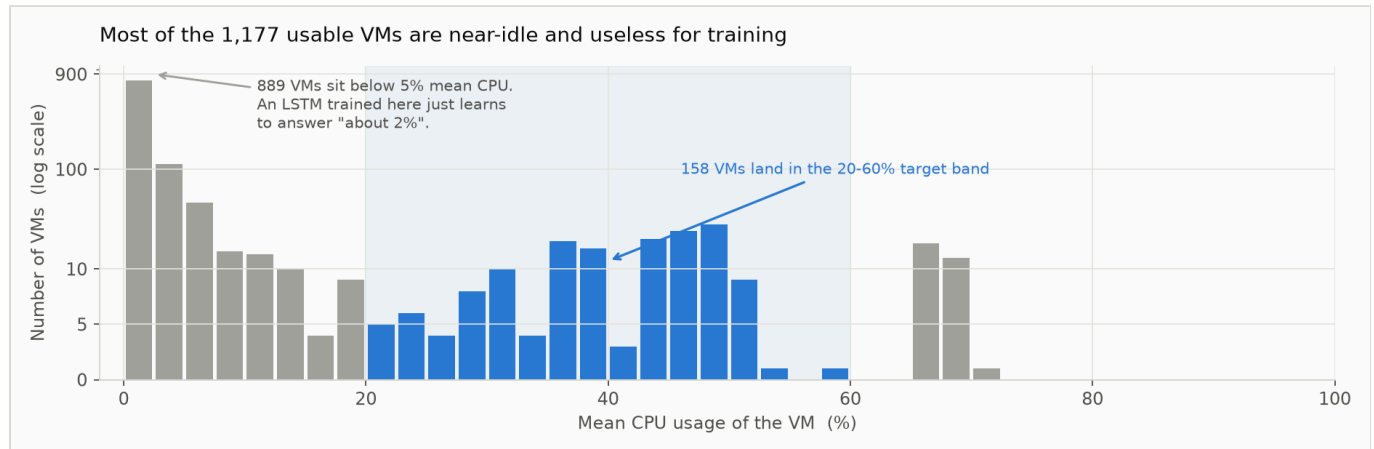
4 Choosing one VM to train on

File: `utils/explore.py` — run with `python -m utils.explore`

In one line: Most servers are almost idle, so we measured all 1,250 and picked a busy one whose load really changes.

Why we could not just take any file

Most VMs in this dataset do almost nothing. Their CPU looks like 2%, 2%, 3%, 2%, 1%... If we train on that, the model learns one lazy answer: "about 2%". It will even get a good-looking error score, because the answer really is almost always 2%. But it has learned nothing useful about spikes — and spikes are the whole point of our project.



Picture. How busy all VMs are. Grey = rejected, blue = in our target range. **889 of 1,177 VMs (77%)** have an average CPU below 5%.

The five rules we used

- 1 Enough data:** at least 2,000 readings (about 7 days). Fewer than that is too short to see any pattern. → 1,250 files became **1,177**.
- 2 Average CPU between 20% and 60%:** below 20% the VM is basically idle; above 60% it is basically always full. Both teach the model "always say the same number". → **158** VMs passed.
- 3 Sort by standard deviation, biggest first:** the VM whose CPU moves the most has the most to learn from.
- 4 Check that all three parts move.** We keep the last 15% of the time as a hidden test set. If the VM goes idle during that period, the test is a flat line and proves nothing.
- 5 Memory and network must not be dead.** The model predicts all three metrics, so a metric stuck on one value would be a column with nothing to learn. A metric is rejected if it never changes, or if one value covers more than 90% of its readings. → caught **12** VMs with dead memory and **225** with dead network across the dataset.

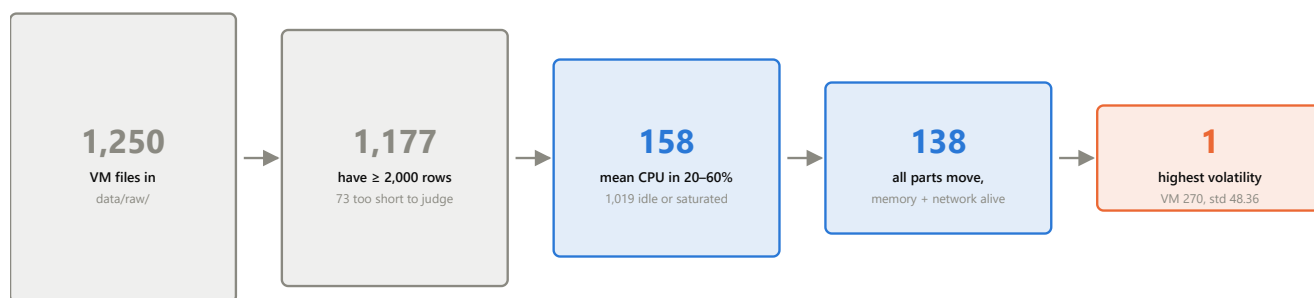
→ **20** rejected by these last checks, leaving **138** real candidates. The winner is **VM 270** (std 48.36).

WHY THE FOURTH RULE EXISTS

We added it after being caught out. With the first three rules the winner was **VM 200**: mean 43%, the highest std of all, no switched-off rows. It looked perfect. But its last 4.5 days are flat zero — and that is exactly the part kept back for testing. Training would have worked and the test score would have been meaningless.

So `explore.py` now measures the three parts the way the model will use them and rejects any VM where one of them does not move. VM 200 is rejected by its own numbers, not by someone noticing.

VM selection funnel — 1,250 candidates down to one training workload

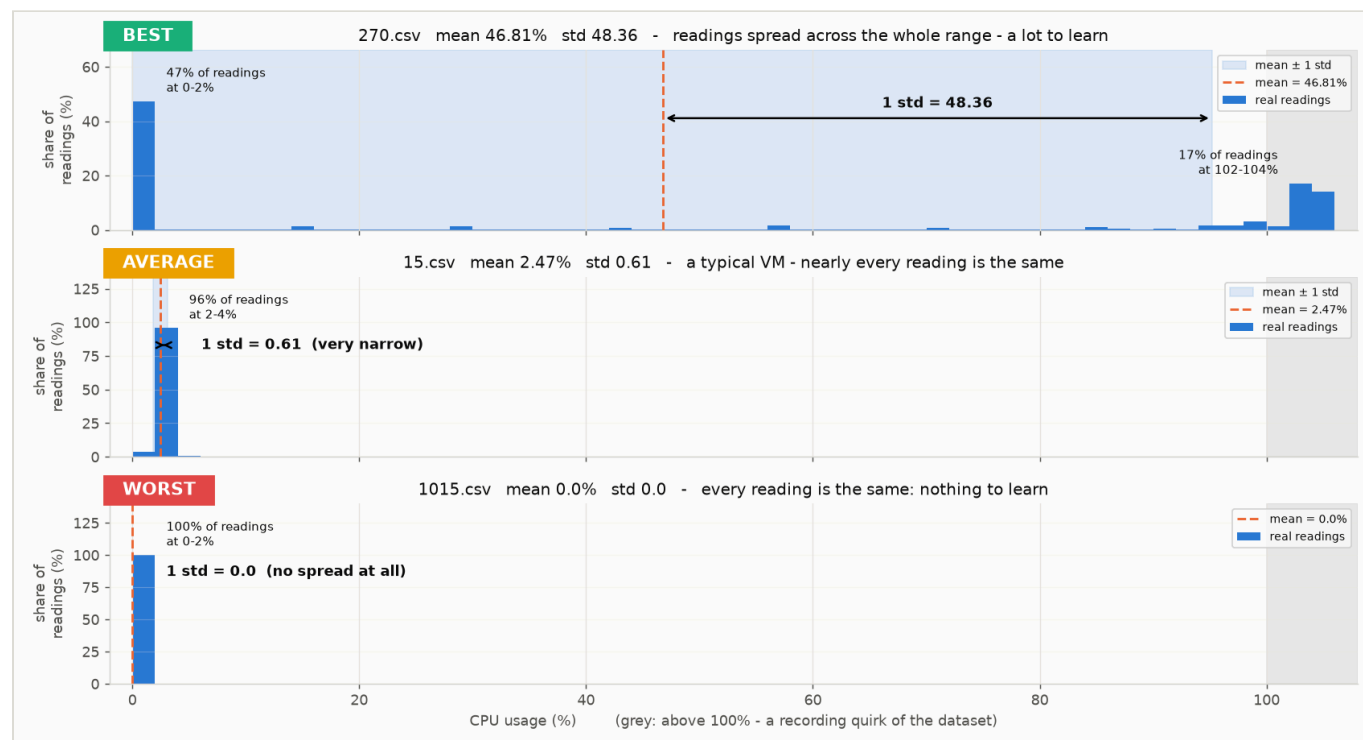


The 20–60% band is not arbitrary: below it the model learns to output a constant near zero, above it a constant near 100.

Ranking by standard deviation asks “does its load actually move?”, and the last check makes sure it keeps moving in the test period too.

Best vs average vs worst — what std looks like

This graph is the easiest way to explain std. The **bars are the real readings**: for each CPU value, how often it actually happened. Drawn on top are the mean (orange line) and the mean $\pm$ 1 std band (blue), so you can see how far the readings normally swing. All three panels use the same x-axis, so the widths can be compared directly.



How to explain it: BEST (VM 270, std 48.36) — readings appear at both ends of the range, so the std band is very wide. There is real movement to learn. **AVERAGE (VM 419, std 0.63)** — a typical VM: 99% of its readings are in one 0–2% bar, so the band is a thin sliver. **WORST (VM 671, std 0.0)** — 100% of readings in one bar: no spread at all.

WHY THERE IS NO BELL CURVE ON THIS GRAPH

A bell curve (Gaussian) is the usual way to draw spread, but it assumes most values sit near the middle.

Our data is not like that, so drawing one would be misleading. We checked VM 270: **53%** of its readings are at 0–2% and **39%** are above 98%. That is **92% at the two extremes**, with the middle almost empty — two peaks, not one hump. Only 60.5% of readings fall within ± 1 std, where a bell curve says 68.3%.

So the graph shows the **real bars**, and std is drawn on top as a band. Mean and std are still correct numbers — they just describe how wide the spread is, not the shape.

Looking at the top VMs over one week

The table cannot show everything, so `explore.py` also draws the top 6 candidates. The left side is one week of CPU with the mean (orange line), the ± 1 std band (blue shading) and the std arrow. The right side is the "average day": the mean CPU for each hour.



Picture. The top 6 candidates. Two things to notice: (1) they look almost identical — probably copies of the same batch job running on several machines, so choosing between them matters little; (2) the "average day" bars **do** show a daily rhythm: quiet in the morning, busiest in the evening. VM 270 averages 29% at 11:00 and 62% at 20:00.

A mistake we found and fixed

Scan all 1,250 files, not the first 150

[all files](#)

FIRST PLAN Check only the first 100–200 files to save time.

WHY CHANGED Files are sorted by name as text, not as numbers: 1.csv, 10.csv, 100.csv, 1000.csv, 11.csv... So "the first 150" is not a fair sample. On this dataset it found **zero** VMs in the 20–60% range.

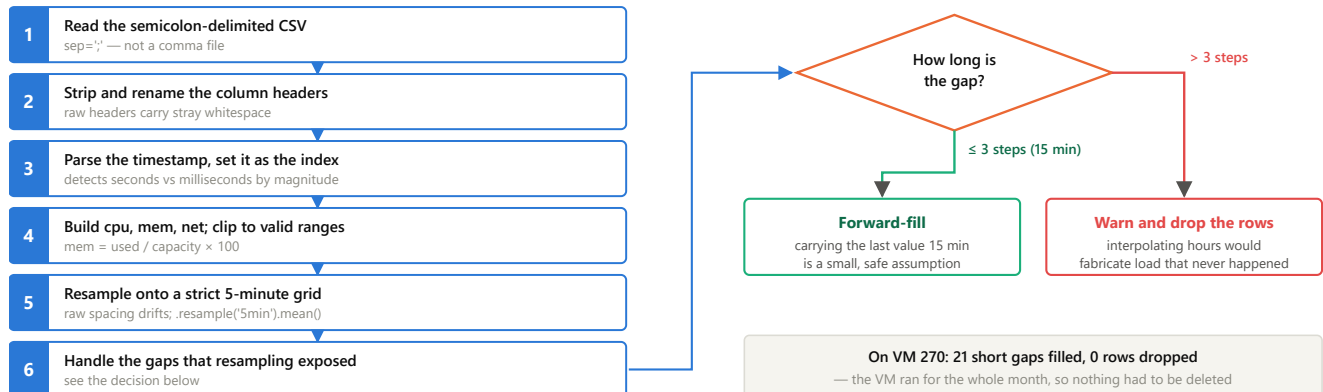
RESULT Scanning everything found 158 in the band and took only about a minute.

5 Cleaning the data

File: `utils/load_data.py` — run with `python -m utils.load_data data/raw/270.csv`

In one line: Turn one messy raw file into a clean table with exactly 4 columns and one row every 5 minutes.

`load_vm()` — six operations, and why the order matters



Each step, and why

Read the file with `;` as the separator

WHY Bitbrains files use semicolons. Reading them as normal comma files would put everything into one column.

Fix the timestamp unit automatically

WHAT The column is named "Timestamp [ms]" (milliseconds), but the numbers are really in **seconds**. The code checks the size of the numbers and picks the right unit.

IF NOT Reading seconds as milliseconds puts all the data in January 1970.

Convert memory to a percentage

WHAT `mem = memory used ÷ memory given × 100`

WHY Raw memory is in KB — numbers in the millions. CPU is 0–100. The model would treat memory as far more important just because its numbers are bigger.

Add network in + network out

WHY For predicting load, the total amount of network activity matters. The direction (in or out) does not.

Clip CPU and memory to 0–100

WHY Some raw readings are slightly above 100% (up to 105.88%). A percentage above 100 is a recording error, so it is set to 100.

Put every row exactly 5 minutes apart (resampling)

WHY Raw readings are not perfectly spaced (sometimes 4 minutes, sometimes 6). The model needs every step to mean the same amount of time.

Handle missing readings: short gaps filled, long gaps removed

[limit: 3 steps](#)

WHAT A gap of up to 15 minutes (3 readings) is filled by copying the last known value. Longer gaps are deleted, never guessed.

WHY Copying a value for 15 minutes is a small, safe guess. Inventing hours of data would teach the model load patterns that never happened.

What happened on VM 270

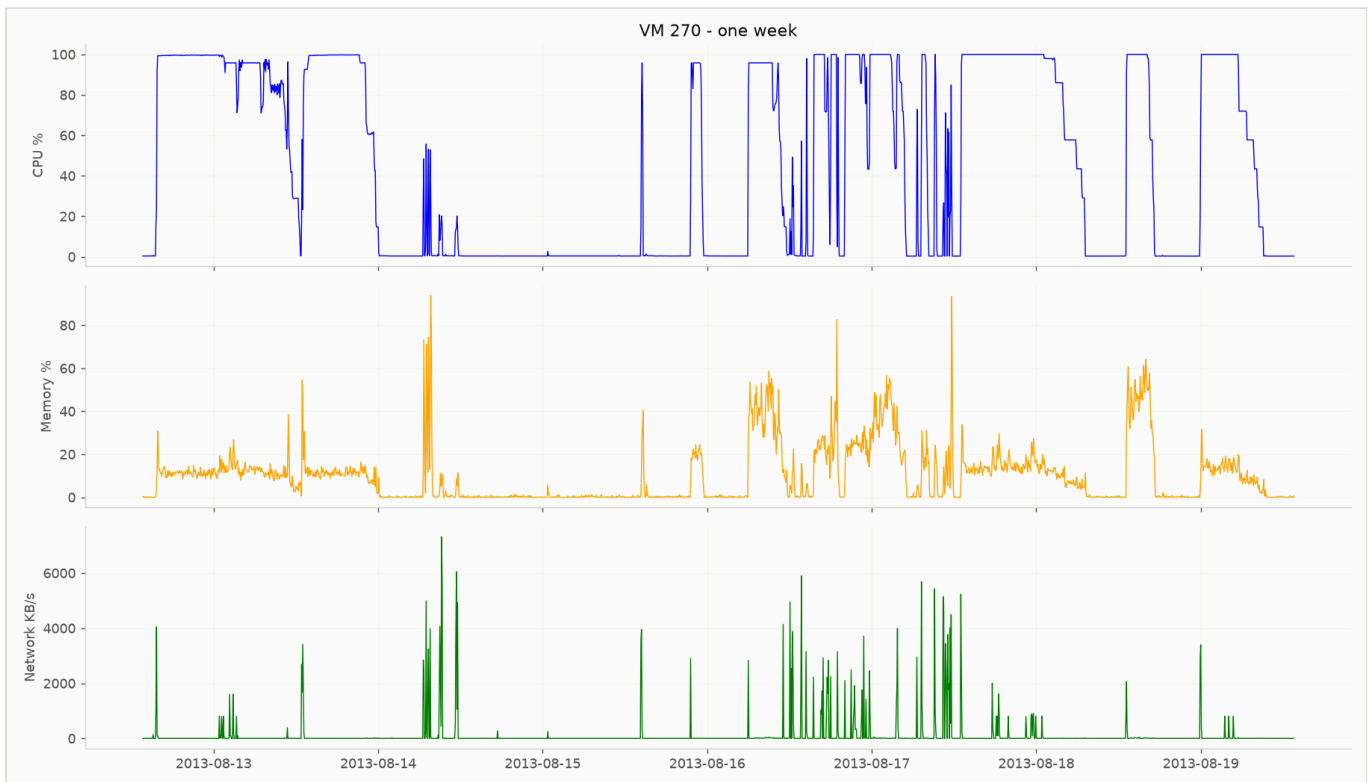
RESULT	VALUE	WHAT IT MEANS
Clean rows kept	8,640	Exactly 30.0 days of data
Short gaps filled	21	Small holes of 15 minutes or less
Rows deleted	0	VM 270 was never switched off
Time jumps left in the data	0	Every row is exactly 5 minutes after the previous one

NOTHING HAD TO BE DELETED — AND WHY THAT MATTERS

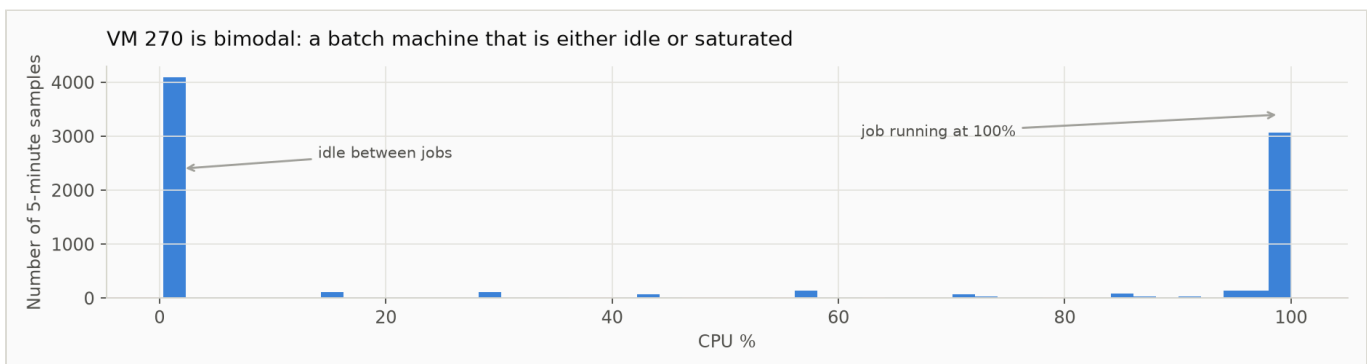
VM 270 ran for the whole month, so no rows were dropped and the clean series has **zero** time jumps: 8,639 steps, every one exactly 5 minutes.

Our first choice, VM 214, was different: 1,329 of its rows had to be deleted because the VM was switched off for its last 4.6 days, with every reading at 0. That discovery is what led us to fix the survey (Part 8), because those zeros were also being counted as "0% busy" when choosing a VM.

The clean result



Picture. One week of clean VM 270 data: CPU (blue), memory (orange), network (green).



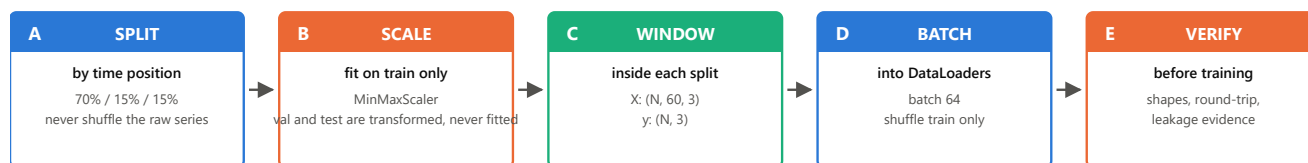
Picture. How often each CPU value appears in the clean data. Almost everything is at 0% or 100%. VM 270 is a **batch machine**: it is either idle or running a job at full power.

6 Preparing the data for the model

File: `utils/preprocessing.py` — run with `python -m utils.preprocessing`

In one line: Split the data by time, scale it, cut it into 5-hour windows, and pack it into batches — *in exactly that order*.

The preprocessing pipeline — five stages whose order is the entire point



IF A AND B ARE SWAPPED, NOTHING CRASHES — AND EVERY METRIC AFTERWARDS IS A LIE

Scaling before splitting lets the test set's minimum and maximum set the transform parameters. The model has then indirectly seen test-set information during training. The original project roadmap has this order wrong; this implementation does not.

WHAT COMES OUT

train_loader

5,988 windows · 94 batches · shuffled

val_loader

1,236 windows · 20 batches · ordered

test_loader

1,236 windows · 20 batches · ordered

scaler.joblib

saved to models/ for the dashboard

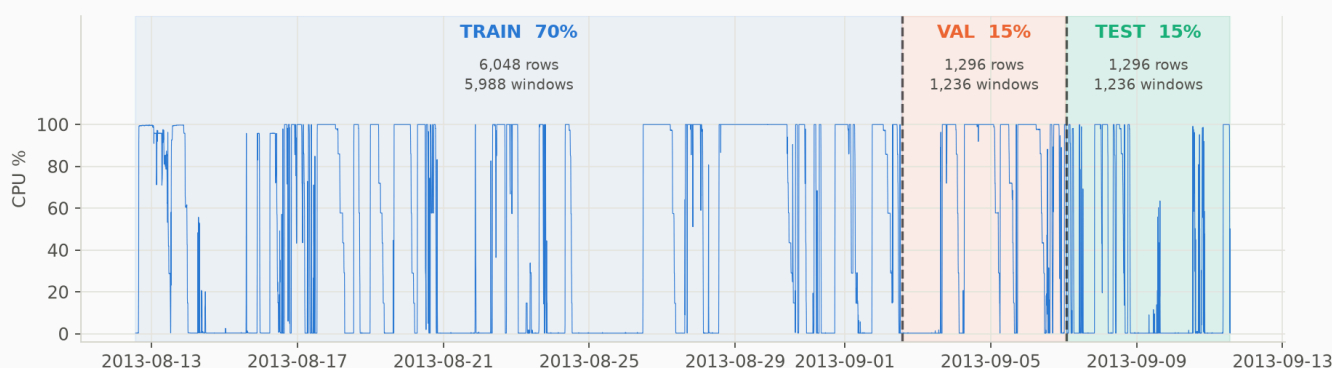
Step A — Split by time: 70% / 15% / 15%

The first 70% of the days is for training, the next 15% for validation, the last 15% for testing. We cut by **position in time**, never randomly.

WHY NOT RANDOM?

In real life we only know the past. If we mixed rows randomly, the model would train on Tuesday and be tested on Monday — it would be "predicting" the past with knowledge of the future. The score would look great and mean nothing.

Step A — the split is by time position, never random



Picture. The split on VM 270's real CPU data. Test data is strictly the last part of the calendar.

Step B — Scale to 0–1 using ONLY the training data

We use `MinMaxScaler`. It learns the smallest and largest value from the **training** data only, then uses those same numbers to scale validation and test.

Why the scaler is fitted on the training split alone

✗ WRONG — scale, then split

```
scaler.fit_transform(whole_series)
```

one scaler fitted over everything

The maximum inside the test set helped decide the transform. The model has now indirectly seen the test data before it was ever evaluated on it.

Symptom: test data also scales to a tidy 0.00–1.00.
The tidiness is the tell.

✓ RIGHT — split, then scale

```
fit_transform(train) · transform(val) · transform(test)
```

scaler fitted here only

unseen

The scaler's parameters come from the past only. Test values are free to fall outside the 0–1 range, and on this dataset they do.

Measured: train 0.00–1.00, test 0.00–1.91.
The overshoot is the proof of no leakage.

HOW WE CAN PROVE THERE WAS NO LEAKAGE

After scaling, training data is exactly 0.00 to 1.00, but test data goes up to **1.91**. That means the test data contains a value higher than anything the scaler saw. This is **good**: it proves the scaler never looked at the test data. If it had, the test data would also fit neatly between 0 and 1.

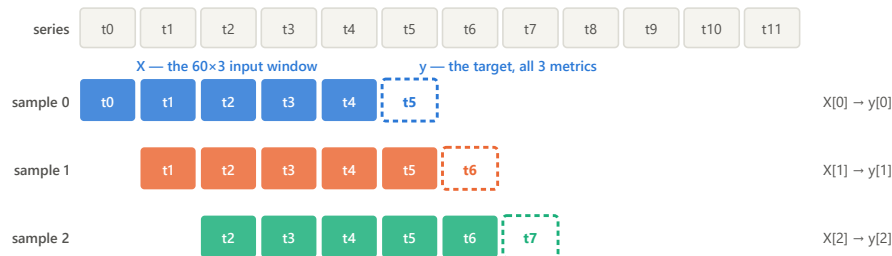
The scaler is saved to `models/scaler.joblib`. Later the dashboard uses it to turn the model's 0–1 answers back into real CPU %, memory % and KB/s.

Step C — Cut into windows (inside each part separately)

Each training example is **60 readings in a row (5 hours)**, and the answer is the **next reading (5 minutes later)**. Then the window moves forward by one step.

Sliding windows: 60 steps of history predict the 61st

Drawn with a window of 5 for legibility; the real window is 60 steps (5 hours) and the real target is 1 step (5 minutes) ahead.



The window slides one step at a time, so consecutive samples overlap by 59 of their 60 steps.

Two consequences: a split of R rows yields exactly $R - 60$ windows (the first 60 rows can only ever be history, never a target), and windowing must run separately per split — one window straddling the train/test boundary would leak the future into training.

Windows are made separately for train, validation and test. A window that started in training data and ended in test data would again leak the future into training.

Step D — Batches

Windows are packed into batches of **64**. The training batches are **shuffled**; validation and test are not.

WAIT — DIDN'T WE SAY NEVER SHUFFLE?

We never shuffle the **readings**. Here we only shuffle the **order of whole windows**. Inside each window the 60 readings are still in the correct time order. Shuffling the order of examples helps the model learn more evenly. Validation and test stay in order so we can plot predictions against real dates.

PART	ROWS	WINDOWS	BATCHES	SHUFFLED?
Training (70%)	6,048	5,988	94	Yes (window order only)
Validation (15%)	1,296	1,236	20	No
Test (15%)	1,296	1,236	20	No

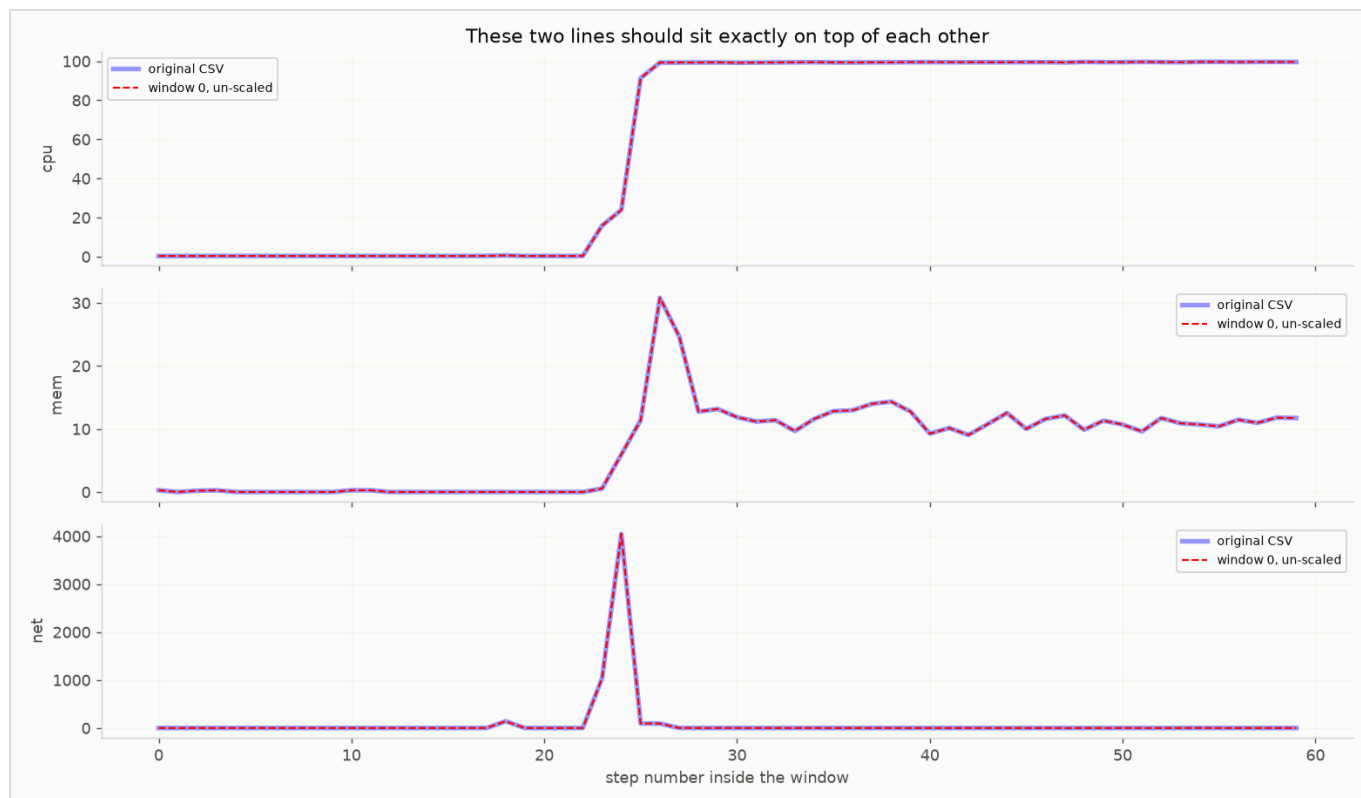
Each part has 60 fewer windows than rows, because the first 60 rows are only used as history.

Step E — Check that everything worked

Mistakes in windowing or scaling do **not** crash the program. They quietly give the model bad data, and you only notice days later when results are poor. So `verify_pipeline()` checks three things before any training:

CHECK	EXPECTED	RESULT
Shape of one batch of inputs	(64, 60, 3)	PASS (64, 60, 3)
Shape of one batch of answers	(64, 3)	PASS (64, 3)
Scale window 0, un-scale it, compare with the original CSV	difference ≈ 0	PASS 0.00024

The tiny 0.00024 difference comes from rounding: PyTorch stores numbers with fewer decimal places (float32) than the CSV (float64). A real bug would give differences of whole units.



Picture. The original data (thick line) and the scaled-then-unscaled data (dashed line) sit exactly on top of each other.

7 Every decision in one place

If someone asks "why did you do it this way?", the answer is here.

About the data

Use real data (Bitbrains)		1,250 VMs
WHY	Real business servers show how load really behaves. Results on real data are more believable.	
IF NOT	Using only made-up data, nobody could be sure the system works in the real world.	

Also build a synthetic (made-up) data generator		planned
WHY	To test proactive vs reactive scaling we need spikes at known times , so we can measure exactly how early we reacted. Real data only has whatever spikes happened.	
IF NOT	The final comparison would depend on luck — whether the real data happened to contain clear spikes.	

Use CPU, memory and network together		3 inputs
WHY	They rise together when a job starts. Seeing all three gives the model more clues.	
IF NOT	A CPU-only model would miss early signs that show up in memory or network.	

Keep the same 4-column format everywhere		datetime, cpu, mem, net
WHY	The real loader and the synthetic generator produce the same table, so every later step works with either one without changes.	
IF NOT	Every later file would need separate code for real and synthetic data.	

About choosing the VM

Minimum readings		2,000
WHY	About 7 days — the least data where a repeating pattern could appear.	
IF NOT	We might pick a VM with only a few hours of data.	

Average CPU range		20% to 60%
WHY	Below 20% = idle, above 60% = always full. Both make the model learn one constant answer.	
IF NOT	The model would score well but predict nothing useful.	

Rank by standard deviation

highest first

- WHY** Std measures how much the CPU moves. Movement is what the model must learn to predict.
- IF NOT** We could pick a VM with a good average but a flat line.

About cleaning

Fill gaps up to 15 minutes, delete longer gaps

3 steps

- WHY** A short copy is a safe guess; long guesses would be fake data.
- IF NOT** Either we lose many rows for tiny holes, or we invent hours of load that never happened.

Readings every 5 minutes

5 min

- WHY** That is how often the dataset was recorded, and it is enough to see spikes building up.
- IF NOT** Uneven spacing would confuse the model about how much time passes between readings.

About preparing for the model

Split by time

70 / 15 / 15

- WHY** The model must learn only from the past and be tested on the future, like in real use. 70/15/15 is a standard three-way split.
- IF NOT** Random splitting leaks future information and gives a fake, too-good score.

Split first, then scale

order A → B

- WHY** The scaler must learn min/max from training data only.
- IF NOT** Test data's values influence training — data leakage. (The original project roadmap had this order wrong; we fixed it.)

MinMaxScaler

range 0–1

- WHY** Neural networks learn more easily when inputs are small and in the same range. CPU and memory already have fixed limits (0–100), which suits a min–max scale.
- IF NOT** Network values (up to about 12,850 KB/s) would dominate CPU and memory.

Window size

60 readings = 5 hours

- WHY** Long enough to see a job start, run and finish. Not so long that training becomes slow and hard.
- IF NOT** Too short: the model misses the build-up. Too long: slower training with little extra benefit.

Predict 1 step ahead

5 minutes

WHY

A new server needs 60–90 seconds to be ready. 5 minutes of warning covers that with room to spare.

IF NOT

A shorter warning would be too late to help.

Batch size

64

WHY

A common size that gives stable learning and fits easily in laptop memory. With 5,988 windows it gives 80 updates per pass through the data.

IF NOT

Very small batches learn noisily; very large ones use more memory.

Predict all 3 values, not only CPU

output 3

WHY

The scaling system needs to know whether a spike is CPU, memory or network, because each may need a different response.

IF NOT

The scaler could only react to CPU spikes.

About the setup

Run everything on a laptop, offline

CPU only

WHY

No cloud account or GPU is needed. Anyone can repeat the work.

IF NOT

Results would depend on paid services.

Do not commit the data to GitHub

.gitignore

WHY

The dataset is about 128 MB; GitHub rejects it. The README will say where to download it instead.

IF NOT

The upload would fail.

8 Honest problems and limits

It is better to say these yourself than to be caught by a question. Each one has a simple explanation.

FOUND AND FIXED 1 — SWITCHED-OFF DAYS WERE BEING COUNTED

When a VM is switched off, a row is still written every 5 minutes with every number at 0. The survey counted those as "0% busy". Our first choice, **VM 214**, was switched off for its last 4.6 days, and those zeros are the only reason it appeared to be inside the 20–60% band:

VM 214	ROWS	MEAN CPU	STD
Counting switched-off rows	11,283	44.21%	49.81
Only while it was running	7,290	68.43%	46.71

68.43% is above our 60% limit, so VM 214 never qualified — and neither did the other five VMs in its group. `explore.py` now skips rows where the memory given to the VM is 0. We checked that rule on a 32-file sample first: in every such row, all the other numbers are 0 too.

FOUND AND FIXED 2 — THE TEST PERIOD WAS NEVER CHECKED

With the first fix in place the new winner was **VM 200**: the highest std of all, never switched off. It looked ideal. But its last 4.5 days are flat zero — and that is exactly the slice we keep back for testing. The model would have trained happily and the final score would have been meaningless.

So the survey now measures train, validation and test separately and rejects any VM where one of them does not move (see Part 4, rule 4). That is how we arrived at **VM 270**.

FOUND AND FIXED 3 — ONLY CPU WAS EVER CHECKED

The first four rules all look at CPU. But the model predicts memory and network too, so a VM could pass every rule while one of those columns is a flat line — and we would be training the model to predict nothing. Rules 5 and 6 now check both, judging each metric against itself (dead = one value covering over 90% of readings) rather than against a fixed band, because memory is a percentage while network is an unbounded KB/s rate.

Measured across the dataset: **12** VMs have dead memory and **225** have dead network. None of them were in our final shortlist, so the chosen VM did not change — the checks are insurance, and they turn a piece of luck into a rule.

STILL OPEN — THE TOP CANDIDATES ARE NEAR-COPIES

VMs 270, 249, 263, 247, 285 and 279 look almost the same, so choosing between them does not really matter. Ranking by std also favours on/off machines: a VM that is half the time at 0% and half at 100% has a std of about 50, the highest possible. VM 270 is still that kind of machine at the 5-minute level — 47% of readings at 0–2%, 36% above 98% — although it does have a clear daily rhythm.

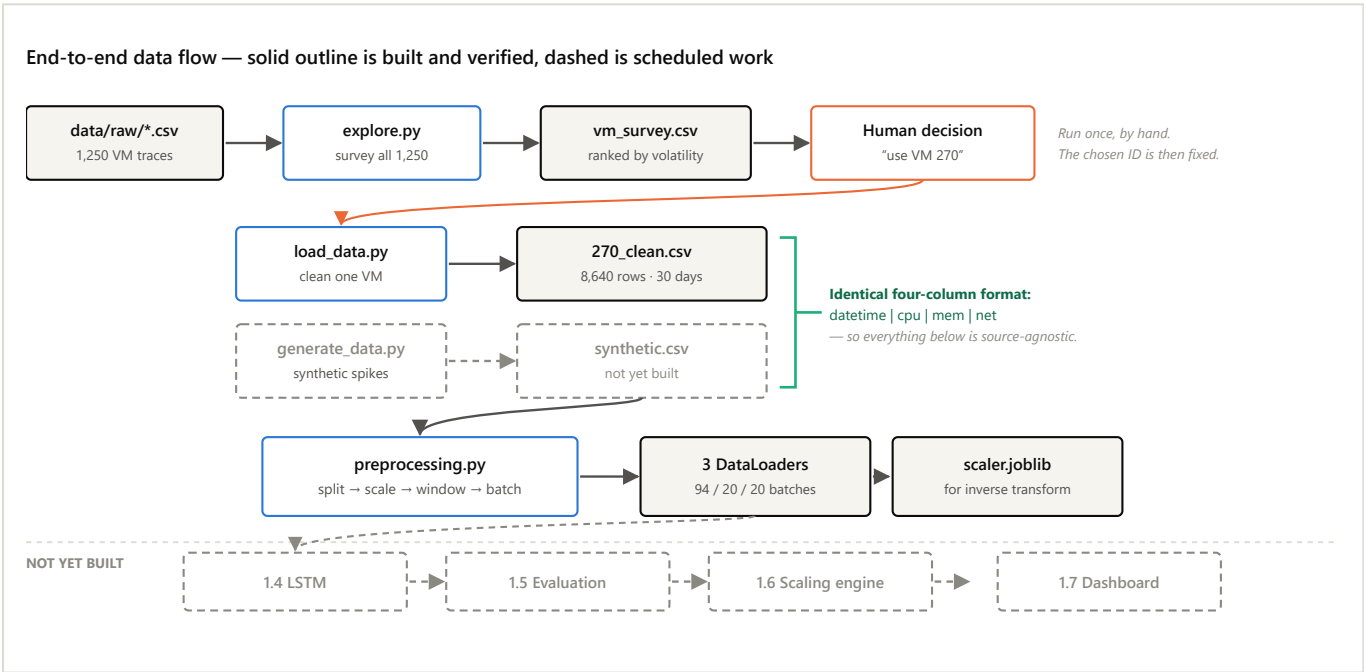
SMALLER LIMITS

- **Only 25 days of data.** Enough for a small model, but not a lot.
- **One VM only.** The model will learn one machine's behaviour, not all servers.
- **Clipping at 100%** hides the difference between 100% and 105%.

Work still missing from Steps 1.1–1.3

ITEM	WHY IT MATTERS
<code>git init</code> + first commit	There is no saved history of the work yet.
<code>README.md</code>	Tells others how to set up and where to download the data. It is the first thing an examiner opens.
<code>utils/generate_data.py</code>	The synthetic data is needed for the controlled spike test in Step 1.6.
<code>notebooks/</code> and <code>dashboard/</code> folders	Part of the planned structure; needed later.

9 The full picture, and what comes next



Where we are

STEP	WHAT	STATUS
1.1	Set up Python environment and folders	MOSTLY DONE
1.2	Get the data, choose a VM, clean it	MOSTLY DONE
1.3	Prepare the data for the model	DONE
1.4	Build and train the LSTM	NEXT
1.5	Measure accuracy and compare with a simple baseline	LATER
1.6	Build proactive vs reactive auto-scaling and compare them	LATER
1.7	Streamlit dashboard	LATER

STEP	WHAT	STATUS
1.8	Put it all together, test, write the report	LATER

What each next step will do, in simple words

- **1.4 LSTM:** the model reads a 5-hour window (60×3 numbers) and outputs 3 numbers: predicted CPU, memory and network 5 minutes later.
- **1.5 Evaluation:** measure how wrong the predictions are (RMSE, MAE, MAPE). Compare with a "naive" guess like "next value = current value". If the LSTM can't beat that, it is not useful.
- **1.6 Scaling engine:** simulate adding servers. Reactive version uses real values; proactive version uses predictions. Compare response time, wasted capacity, and false alarms.
- **1.7 Dashboard:** a web page showing predicted vs actual load and when servers were added.
- **Phase 2:** put a 4-qubit quantum circuit between the LSTM and the final output layer, and compare with Phase 1.

10 Questions your teacher may ask

Q. What does your project do, in one sentence?

It predicts server load 5 minutes ahead with an LSTM so that extra servers can be started before a traffic spike, instead of after it.

Q. Why is reactive scaling not good enough?

A new server takes 60–90 seconds to be ready. Reactive scaling only starts after the spike is seen, so users always face that delay.

Q. Why did you use this dataset?

It is real data from business servers (banks, insurers), openly published by TU Delft, and widely used in cloud research.

Q. Why not just use the first VM file?

76% of VMs are almost idle. A model trained on them just predicts "about 2%" and learns nothing about spikes. So we measured all 1,250 and chose a busy, changing one.

Q. What is standard deviation and why did you use it?

It measures how far values spread from the average. A high std means the CPU changes a lot, which is what the model needs to learn. The best/average/worst graph shows this: wide curve vs thin spike vs flat line.

Q. Is VM 270's CPU normally (Gaussian) distributed?

No, and we checked rather than assuming. 53% of readings are at 0–2% and 39% above 98% — 92% at the two extremes, so the shape has two peaks, not one hump. Only 60.5% fall within ± 1 std (a bell curve says 68.3%). That is why our graph shows the real readings as bars instead of a bell curve.

Q. Is VM 270 really the best choice?

Yes, after fixing two faults in the survey. The first choice, VM 214, only looked valid because its switched-off days were counted; while running it averaged 68%, outside our band. The next winner, VM 200, had a flat test period. VM 270 passes all four rules: 30 full days, never switched off, and all three parts move. (Part 8.)

Q. Why convert memory to a percentage?

Raw memory is in millions of KB and CPU is 0–100. Without converting, the model would think memory is more important just because the numbers are bigger.

Q. You deleted 1,329 rows. Is that a problem?

That was VM 214, our first choice. Those rows were all at the end of the file, when the VM was switched off and every reading was 0. VM 270, the VM we actually use, needed **no** rows deleted at all and has zero time gaps.

Q. What is data leakage and how did you prevent it?

Leakage is when test information sneaks into training. We prevent it three ways: split by time first; fit the scaler on training data only; make windows separately in each part. Proof: test data scales up to 1.91, outside 0–1, which only happens if the scaler never saw it.

Q. Why shuffle training data if it is a time series?

We shuffle only the order of whole 5-hour windows. The readings inside each window stay in time order.

Q. Why a window of 60?

60 readings × 5 minutes = 5 hours. That is long enough to see a job start and finish, but short enough to train quickly.

Q. How do you know the pipeline has no bugs?

`verify_pipeline()` checks batch shapes (64, 60, 3) and (64, 3), and un-scales a window to compare it with the original data. The difference is 0.00024, which is just rounding.

Q. Where will the quantum part go?

Between the LSTM's output and the final layer that makes the 3 predictions. Everything else stays the same, so Phase 1 is a fair comparison.

11 How to run each part

Examiners often say "show me this part running". Every file can run on its own.

SETUP (ONCE)

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Download the Bitbrains fastStorage data and unzip the CSV files into `data/raw/`.

1. SURVEY ALL VMS AND DRAW THE COMPARISON GRAPHS

```
python -m utils.explore          # shows the top 6
python -m utils.explore 10       # shows the top 10
```

You will see: a ranked table, and two saved pictures — `plots/top_vms.png` and `plots/cpu_distribution.png` (best / average / worst). Takes about 1–2 minutes.

2. CLEAN THE CHOSEN VM

```
python -m utils.load_data data/raw/270.csv
```

You will see: rows kept, gaps fixed, the first rows, statistics, and a one-week plot saved to `plots/270_week.png` . The clean file is saved to `data/processed/270_clean.csv` .

3. PREPARE AND CHECK THE DATA FOR THE MODEL

```
python -m utils.preprocessing
```

You will see: batch shapes, the reconstruction check, the row/window counts, and the scaled ranges. It saves `models/scaler.joblib` and `plots/verify_pipeline.png` . Takes a few seconds.

Files at a glance

FILE	JOB
<code>utils/explore.py</code>	Measure all VMs, rank them, draw comparison graphs
<code>utils/load_data.py</code>	Clean one raw VM file into 4 columns
<code>utils/preprocessing.py</code>	Split, scale, window, batch, and verify
<code>data/processed/vm_survey.csv</code>	The ranked table of all VMs
<code>data/processed/270_clean.csv</code>	Clean data for VM 270
<code>models/scaler.joblib</code>	Saved scaler for turning predictions back into real units